Machine Learning — Exam Notes

1. Introduction to Machine Learning

- **ML**: field of study giving computers the ability to learn without being explicitly programmed; evolved from Data Mining (KDD).
- **KDD Process**: Data → Preprocessing → Data Mining/ML → Pattern Evaluation → Knowledge.
- **Supervised Learning**: learn from labeled data.
  - *Classification*: predict a discrete/categorical label (e.g., spam/not spam).
  - *Regression*: predict a continuous value (e.g., price).
- **Unsupervised Learning**: learn from unlabeled data.
  - *Clustering*: find natural groups (e.g., customer segments).
  - *Association Rule Mining*: find relationships between items (e.g., market basket analysis).
- **Classification vs Regression**: output type is the key difference — categorical vs. continuous.
- **Key Challenges**: scalability, high dimensionality, complex/heterogeneous data, data quality, data ownership/distribution, privacy preservation, streaming data.

---

2. Data & Data Preprocessing

- **Data set** = collection of data objects (a.k.a. records/points/instances/samples/entities), each described by **attributes** (a.k.a. features/variables).
- **Types of data sets**: Record, Graph, Ordered (sequential/temporal).
- **Attribute types**: Nominal, Ordinal, Interval, Ratio (also grouped as Categorical vs. Numeric; Discrete vs. Continuous).
- **Data Quality dimensions**: Accuracy, Completeness, Consistency, Timeliness.
- **Four major preprocessing tasks**:
  1. **Cleaning** — handle missing/noisy/inconsistent data.
  2. **Integration** — merge data from multiple sources.
  3. **Transformation** — normalization, aggregation, discretization.
  4. **Reduction** — dimensionality/feature reduction, sampling.

---

3. Foundations of Supervised Learning / Classification

- **Classification**: build a model on labeled training data; use it to predict labels for unseen data.
- **Train/Test split**: model learns on training data, is evaluated on held-out test data to estimate generalization.
- **Evaluation Metric — Accuracy**: ratio of correctly classified instances to total instances.

$$Accuracy = \frac{\text{Correct predictions}}{\text{Total predictions}}$$

- **Overfitting**: model fits training data (incl. noise) too closely → poor generalization to new data.
- **Underfitting**: model is too simple to capture the underlying pattern → poor performance on both train and test data.

---

4. Decision Trees

- Tree-structured classifier: internal nodes = attribute tests, branches = outcomes, leaves = class labels.
- **Splitting**: at each node, choose the attribute/split that best separates classes (best "purity" gain).
- **Impurity Measures**:
  - **Entropy**: $Entropy(t) = -\sum_j p(j|t) \log_2 p(j|t)$; **Information Gain** = parent entropy − weighted average child entropy.
  - **Gini Index**: $Gini(t) = 1 - \sum_j [p(j|t)]^2$
    - Gini = 0 → node perfectly pure.
    - For a **two-class 50/50 split**, Gini = **0.5** (maximum impurity).
    - **Gini of a split** = weighted average of children's Gini indices: $Gini_{split} = \sum_i \frac{n_i}{n} Gini(N_i)$
  - **Misclassification Error**: another (less commonly used) impurity measure.
  - Goal for all three: pick the split that gives the **largest reduction** in impurity.
- **Handling attribute types**: binary split for continuous/ordinal attributes (threshold-based); multi-way or binary split for categorical/nominal attributes.
- **Pruning**: remove branches that add little predictive power, to reduce overfitting (pre-pruning: stop early; post-pruning: grow full tree then trim).
- **Pros**: interpretable, handles mixed data types, no need for feature scaling.
- **Cons**: prone to overfitting without pruning, can be unstable (sensitive to small data changes).

---

5. Rule-Based Classification

- Classify using a set of IF-THEN rules (antecedent → consequent/class).
- **Rule Generation**:
  - **Direct Method**: extract rules directly from data (e.g., sequential covering/ RIPPER).
  - **Indirect Method**: derive rules from another model, typically a decision tree (each root-to-leaf path → one rule).
- **Rule Assessment**:
  - **Coverage**: fraction of the dataset satisfying the rule's antecedent.

$$Coverage = \frac{n_{covers}}{|D|}$$

- **Accuracy**: fraction of covered tuples that are correctly classified by the rule.

$$Accuracy = \frac{n_{correct}}{n_{covers}}$$

- High coverage + high accuracy = a strong, reliable, broadly-applicable rule; low accuracy = unreliable even if it applies often; low coverage = rule is narrow/specific even if precise.

- Rules can conflict or leave records uncovered — need conflict-resolution (e.g., rule ordering) and default rules.

---

6. kNN (k-Nearest Neighbors) — Lazy Learning

- **Eager vs. Lazy Learning**: Decision Trees/Rules = eager (build a generalized model upfront). kNN = **lazy** — stores training data and defers computation to prediction time.
- **Algorithm**: for a new instance, find the *k* closest training points (by a distance metric, e.g., Euclidean) and assign the majority class among them (or average, for regression).
- **Feature Scaling is essential for kNN**: attributes with larger numeric ranges dominate the distance calculation and can wash out the contribution of more discriminative but smaller-scale features.
  - **Min-Max Normalization**: rescales values to a fixed range (e.g., [0,1]).
  - **Standardization (Z-score)**: rescales to mean 0, standard deviation 1.
- **Choosing k**: small k → sensitive to noise (overfitting); large k → smoother boundary but may include irrelevant neighbors (underfitting/oversmoothing).
- **Pros**: simple, no training phase, naturally handles multi-class.
- **Cons**: computationally expensive at prediction time, sensitive to irrelevant features/scale, sensitive to k choice.

---

7. Ensemble Learning

- **Idea**: combine multiple ("weak") models to get a more accurate, robust prediction than any single model — "wisdom of the crowd."
- **Bagging (Bootstrap Aggregating)**: train multiple models independently on bootstrap samples (random sampling with replacement) of the data; aggregate by majority vote/averaging. Reduces variance.
- **Random Forest**: bagging of decision trees + random feature subset selection at each split → decorrelates trees, builds an ensemble of diverse trees.
- **Boosting**: builds models **sequentially**, each new model focuses on correcting errors of previous ones. Reduces bias.
- **AdaBoost**: a boosting algorithm that iteratively reweights misclassified instances so subsequent learners focus more on hard cases; final prediction = weighted vote of all weak learners.

---

8. Regression

- **Regression** models the relationship between input feature(s) *x* and a continuous target *y*.
- **Linear Regression**: $\hat{y} = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n$
- **Cost Function** (typically Mean Squared Error):

$$J(\theta) = \frac{1}{2m} \sum_{i=1}^{m} (\hat{y}^{(i)} - y^{(i)})^2$$

- **Gradient Descent**: iterative optimization to minimize the cost function by updating parameters in the direction of steepest descent:

$$\theta_j := \theta_j - \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

  where α = learning rate.

---

9. Clustering — Foundations

- **Clustering**: given data points, group them so points within a cluster are more similar to each other than to points in other clusters (unsupervised — no labels).
- **Types of clustering**:
  - Partitional (flat, single-level, e.g., K-means) vs. Hierarchical (nested, tree-like).
  - Exclusive (each point in one cluster) vs. Overlapping/fuzzy.
- **Applications**: document clustering, stock market grouping, market segmentation.

---

10. K-Means Clustering

- **Definition** (Lloyd, 1957): partition N points into K disjoint clusters minimizing the **sum of squared error (SSE)** between points and their cluster centroid.
- **Algorithm**:
  1. Choose K initial centroids (often randomly).
  2. Assign each point to nearest centroid.
  3. Recompute centroids as mean of assigned points.
  4. Repeat 2–3 until convergence (centroids stabilize / assignments don't change).
- **Objective function**: $SSE = \sum_{i=1}^{K} \sum_{x \in C_i} dist(x, c_i)^2$ — algorithm converges to a local minimum.
- **Challenges**:
  - Sensitive to initial centroid choice → can converge to poor local optima (mitigate with multiple runs / k-means++).
  - Must pre-specify K.
  - Sensitive to outliers and works best on globular (spherical), similarly-sized clusters.
- **Choosing K**: Elbow method (plot SSE vs K, look for the "elbow"), Silhouette score,

domain expertise.

---

11. Hierarchical Clustering

- Builds a hierarchy/tree (**dendrogram**) of clusters — no need to pre-specify K.
- **Agglomerative (bottom-up)**: start with each point as its own cluster, repeatedly merge the two closest clusters until one cluster remains.
- **Divisive (top-down)**: start with one cluster containing all points, repeatedly split until each point is its own cluster.
- **Linkage criteria** (how to measure distance between two clusters):

Method	Definition	Strength	Weakness
Single Linkage (MIN)	Distance = shortest distance between any point in A and any point in B	Finds non-globular/ complex shapes	Sensitive to noise → "chaining effect"
Complete Linkage (MAX)	Distance = longest distance between any point in A and any point in B	Robust to noise, balanced cluster sizes	Biased toward globular clusters; can break large natural clusters
Average Linkage	Distance = average of all pairwise distances between points in A and B	Compromise between MIN and MAX; more robust than single, less biased than complete	Computationally more expensive

- **Process**: build/update a **proximity (distance) matrix** at each step, merge the closest pair of clusters per the chosen linkage, repeat until a single cluster/dendrogram is formed.
- For dataset like **two concentric circles**: use a linkage/method sensitive to shape rather than centroid distance — e.g., **single-linkage agglomerative** (or density-based DBSCAN) — since K-means and complete/average linkage assume globular clusters and would fail here.

---

12. DBSCAN & BIRCH

- **Motivation**: K-means fails on non-globular clusters; hierarchical clustering is computationally expensive for large data.
- **DBSCAN (Density-Based Spatial Clustering)**:
  - Groups points that are closely packed (high density), marks low-density points as **noise/outliers**.
  - Key parameters: **eps** (neighborhood radius), **MinPts** (minimum points to form a dense region).
  - Point types: **Core point** (⩾ MinPts within eps), **Border point** (within eps of a core point but < MinPts itself), **Noise point** (neither).

    - Pros: finds arbitrarily shaped clusters, robust to outliers, no need to specify K.
    - Cons: struggles with varying density clusters, sensitive to eps/MinPts choice.
- **BIRCH (Balanced Iterative Reducing and Clustering using Hierarchies)**:
    - Designed for **scalability** on very large datasets.
    - Builds a **CF-Tree (Clustering Feature Tree)** — a compact, in-memory summary of the data (each CF = count, linear sum, squared sum of points in a sub-cluster).
    - Single scan of data suffices to build the CF-tree; then clustering is applied on the (much smaller) CF-tree leaves.

---

13. Association Rule Mining & Apriori

- **Goal**: discover relationships/co-occurrence patterns between items in large transactional datasets (e.g., "customers who buy bread also buy butter") — different from clustering (which groups similar records).
- **Key measures**:
    - **Support**: fraction of transactions containing an itemset. $Support(X) = \frac{\text{transactions containing } X}{\text{total transactions}}$
    - **Confidence**: how often rule $X \to Y$ holds. $Confidence(X \to Y) = \frac{Support(X \cup Y)}{Support(X)}$
- **Apriori Algorithm**:
    - Based on the **Apriori property**: all subsets of a frequent itemset must also be frequent (anti-monotonicity) — used to prune the search space.
    - Iteratively generates candidate itemsets of increasing size, prunes those below a minimum support threshold, and repeats until no larger frequent itemsets remain.
    - Frequent itemsets are then used to generate rules meeting a minimum confidence threshold.
- **Applications**: market basket analysis, supermarket shelf placement, inventory management, cross-selling.

Quick-Reference Formula Sheet

Concept	Formula
Accuracy	Correct / Total
Entropy	$-\sum p(j\|t) \log_2 p(j\|t)$
Gini Index	$1 - \sum [p(j\|t)]^2$
Gini of split	$\sum_i \frac{n_i}{n} Gini(N_i)$
Rule Coverage	$n_{covers} / \|D\|$
Rule Accuracy	$n_{correct} / n_{covers}$
K-means objective (SSE)	$\sum_i \sum_{x \in C_i} dist(x, c_i)^2$
Linear regression cost	$\frac{1}{2m} \sum (\hat{y} - y)^2$
Support	$Support(X \cup Y)$
Confidence	$Support(X \cup Y) / Support(X)$